



AdBuster 2.0 PRO

Technical Project Monograph

Architecture, Audio Analysis, CEPA, Machine Learning
and Prototype Development

Windows prototype | Public PRO test build

English edition | September 2026

Compiled package; source code is not included

D.P-G / AdBuster Development Team

Contents

Introduction 3

 Aim, questions and method 3

 1. Project summary 3

 2. Problem and prototype objective 3

 3. Project versions and scope of this publication 4

 4. System architecture 4

 5. Audio analysis and loudness patterns 5

 6. CEPA: context and decisions 5

 6.1 A classifier and CEPA answer different questions 6

 6.2 Context and events over time 7

 6.3 Decision layers 7

 6.4 Zones, dead zone and anti-drift 7

 6.5 Worked decision example 7

 6.6 Originality: what the evidence supports 8

 7. Machine learning and data 8

 8. Broadlink, VolMaster and IR control 9

 9. Privacy and local operation 9

 10. Testing, results and evidence limits 10

 10.1 Evidence basis 10

 10.2 Proposed validation protocol 10

 11. Prototype limitations 10

 12. Startup and requirements 11

 13. Development status 11

 14. Licensing, feedback and documentation 11

 15. Project graphics and screenshots 11

 16. Glossary 17

 17. Project source materials 18

Introduction

Scope of this publication This monograph consolidates the project documents supplied by the author. It describes the public AdBuster 2.0 PRO test package separately from the research materials for Hybrid Engine v13 and the experimental AD Detector. It is an explanatory project publication, not a user manual, independent benchmark, warranty, or replacement for the licenses.

Aim, questions and method

This monograph examines how AdBuster connects acoustic observation of television audio to a decision about a possible volume correction, and explains CEPA's place between signal analysis, ML and IR actuation. It addresses three questions:

- 1 Which layers turn an audio measurement into a constrained command to a television?
- 2 What role does CEPA perform beyond AD/NORMAL classification, and which elements can be supported as an architectural differentiator?
- 3 What do the documentation and screenshots establish about readiness, and what do they leave unverified?

The method is a critical analysis of project documents: the README, package inventory, architecture descriptions, licenses, laboratory materials, reports, logs and screenshots. Statements are separated by evidence status: a file listed in the package, a feature claimed in documentation, an observation in an individual log or screenshot, and a concept shown only in a diagram. This is documentary analysis, not a source-code audit, systematic literature review or independent experiment.

1. Project summary

AdBuster 2.0 PRO is a Windows prototype exploring whether TV volume can be stabilized by analysing sound picked up by a computer microphone. The application observes audio measurements, interprets how they change over time and, when a compatible Broadlink device is configured, can send infrared commands to adjust the TV volume.

The project brings together real-time audio analysis, CEPA decision logic, machine-learning components and an IR control layer. These describe prototype and development work. Terms such as “AI” and “human-like” express the design approach; they are not proof of effectiveness or human understanding of audio content.

The public PRO package is compiled and does not include source code. It is made available for testing and evaluation. The project is not published as open source.

2. Problem and prototype objective

Viewers may experience abrupt changes in perceived loudness between television programmes and commercials. AdBuster explores whether a nearby Windows computer can monitor sound through a microphone and make limited volume corrections through an external IR transmitter.

The prototype does not analyse video, subtitles or a broadcaster's digital stream. It does not need to know the source of the sound: it analyses the acoustic signal reaching the microphone. It therefore does not identify advertisements by meaning. It looks for sound patterns that a model or rules may treat as advertisement-like or as unusual loudness changes.

The project objective is practical and exploratory: combine these measurements with cautious volume control, then observe whether the resulting behaviour is predictable in different conditions. This remains a prototype; results may depend on the microphone, room, TV, settings and Broadlink device.

3. Project versions and scope of this publication

The project files describe several stages that can be easy to conflate. This monograph separates them as follows:

- **Early proof of concept** — the [project README](#) retrospectively describes the first prototype as a rough script on older hardware, without a user interface, machine learning, CEPA or visualisation. The supplied sources do not date this stage, so this monograph assigns no year or release number to it.
- **Public AdBuster 2.0 PRO Test Build** — distributed as a ZIP. Its package inventory lists, among other items, `AdBuster.exe`, `Aduster.exe`, `VolMaster.exe`, `Start.bat`, configuration files, `model.pkl` and `model_deep.pkl`. The current [README](#) describes the package as including CEPA Core in a Perception → Behaviour → Action structure.
- **Full CEPA Logic v2** — the [README](#) distinguishes this from the CEPA Core in the public build and states that the full version described in the documentation is not included in the test ZIP. This matters: diagrams of the more extensive logic do not prove that every described mechanism is present in package 2.0.
- **Hybrid Engine v13** — a separate laboratory track documented in `labs/hybrid`, with its own diagrams, settings and startup report. A v13 description should not automatically be read as a description of every feature in the public 2.0 ZIP.
- **Experimental AD Detector** — `ad_detector.pkl` and its `data.csv` are described in laboratory/development materials, outside the public 2.0 ZIP.

This version boundary matters. The repository contains public package documentation, historical material and experiments. Statements about the current ZIP refer to its inventory and public-build description. References to v13 or `ad_detector.pkl` identify separate research material.

The history is best represented as documented development landmarks, not a simple linear sequence in which each named track replaces the previous one: early audio measurement, the public 2.0 build with CEPA Core and models, the separately described full CEPA Logic v2, and the Hybrid v13 and AD Detector experiments. The sources identify these landmarks but do not provide a complete, dated history of every release.

4. System architecture

At a functional level, the system can be described as four cooperating parts. These are roles, not necessarily individual executable files.

- 1 **Audio input and analysis** — a microphone supplies audio samples. The application calculates simplified measurements and observes how they change over time.
- 2 **Classification and context** — ML components can provide labels or cues about audio behaviour. CEPA combines these with context, history and reaction constraints.
- 3 **Application logic and safety** — determines whether to remain idle, send a correction, wait or block an action due to limits.
- 4 **IR layer** — `VolMaster`, documented as a local Flask server, forwards a command to a Broadlink device, which sends an infrared signal to the TV.

The ZIP inventory lists several separate helper applications. `Start.bat` is used to launch the packaged components. The logical architecture should therefore not be confused with the number of EXE files.

The diagram shows component roles rather than a strict function-call graph. In the user-facing path, observation begins acoustically: a microphone captures sound in the room, not the television's digital broadcast stream.

Processing and control can be described as follows:

- 1 **Microphone** captures TV sound in the room.
- 2 **DSP / audio engine** calculates the current level and features or measurement history; details depend on the variant.
- 3 **ML, when enabled in a given mode**, can provide an AD/NORMAL label or cue. The model is not an IR transmitter.
- 4 **CEPA** interprets change in the context of events, state and recent history; depending on the version, it selects no action or a candidate correction.

- 5 **Application logic and safety layer** apply limits such as cooldown, anti-spam and command caps.
- 6 **VolMaster** receives a local HTTP request and forwards the command to a Broadlink device.
- 7 **Broadlink** sends IR to the television. The microphone captures the changed room sound again, forming an acoustic observation loop; the documentation does not describe volume telemetry read back from the TV.

This is a functional model. Auto and ML modes, different packages and laboratory tracks may divide responsibilities differently or omit some stages.

5. Audio analysis and loudness patterns

Project documents describe measurements such as RMS, short-term changes, deviation, range, trends, spikes and stability. These are numerical signal measurements, not transcription or semantic analysis of conversation. They can be used to observe whether sound is rising, remaining steady, changing abruptly or returning to an earlier range.

The documents describe different feature sets at different stages. The acoustic report discusses 29 features, including MFCC, Chroma, spectral centroid, bandwidth, RMS and ZCR. Other materials for the hybrid model describe a reduced four-feature set: RMS, STD, DELTA and RANGE. These figures should not be combined into a single model description without identifying the version and ML path.

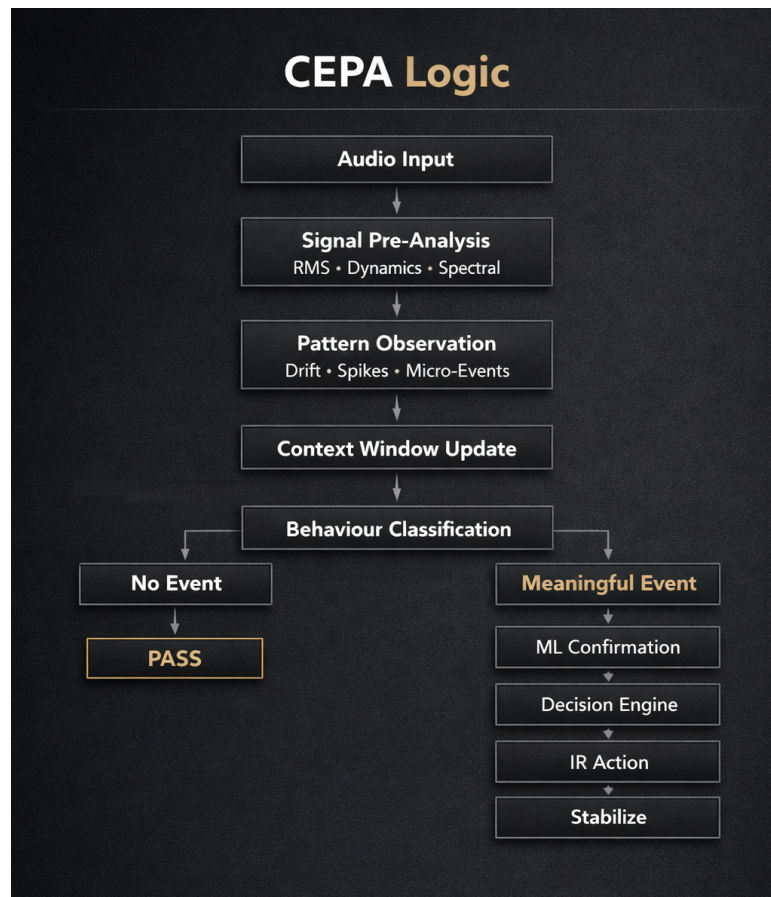
Project files use working categories such as AD, NORMAL, MUSIC and DIALOG, as well as drift, spike, stable zone and transition. These labels describe signal behaviour. They do not mean that the system will always identify the type of content correctly in every scene.

6. CEPA: context and decisions

CEPA stands for **Contextual Event Pattern Analysis**. In this project, the name refers to a decision layer intended to interpret sequences of changes instead of reacting to every individual increase in level. A high reading could be a short sound effect, music or a change during a commercial; timing, stability and previous system behaviour also matter.

Conceptual materials describe CEPA as combining context observation, pattern detection, action selection and reaction limits. Documented possible outcomes include VOL_DOWN, VOL_UP, PASS, and blocking or delaying an action. Details differ between version descriptions and are not presented here as a complete implementation specification for the public ZIP.

“Human-like” is used in the project documentation as shorthand for goals such as ignoring small fluctuations, avoiding overly frequent corrections and considering recent history. It does not mean the algorithm is conscious, understands advertisements or reproduces human reasoning.



CEPA decision flow

The diagram separates a “No Event” branch, which ends in PASS, from an event treated as meaningful, which proceeds to ML confirmation and a further decision. IR action and stabilisation follow. Its central idea is filtering before actuation: an acoustic change does not immediately become a command. The image summarises a conceptual model; the public build contains CEPA Core, and the fuller diagram does not prove that every shown step is implemented.

The following subsections explain CEPA’s role as a decision layer. They combine the public CEPA Core outline with the more extensive logic materials; they distinguish a concept or full version from a confirmed feature of the public 2.0 ZIP.

6.1 A classifier and CEPA answer different questions

A classical classifier primarily answers: which class best matches the analysed feature window, for example AD or NORMAL? Its output may be a label or probability. That result alone does not determine whether to send a command to the TV, how long to wait, or whether a previous correction was sufficient.

CEPA is intended to solve a different problem: **which response should be allowed for an event in the current context?** It can combine an ML result with trend, stability, transition, previous actions, user mode and constraints. In this design, classification is one input signal rather than a direct VOL_DOWN instruction.

- **ML:** recognises a pattern from features and training data; its behaviour depends on the model, labels and data distribution.
- **CEPA:** maintains state and response rules over time; it can return PASS, delay an action or permit a limited correction.
- **Application layer:** handles execution, cooldowns, anti-spam and command limits.

Separating these roles makes failure analysis clearer: a classifier error, a contextual decision and a failed IR transmission are different events. This separation does not by itself prove that the classifier is more accurate or that CEPA always avoids false reactions.

6.2 Context and events over time

A single level measurement does not describe an entire passage. The same value may occur during stable dialogue, a short effect, a gradual rise or a sustained loud segment. CEPA materials therefore propose interpreting temporal relationships: direction and rate of change, repetition, stability and return to an earlier range.

The full-logic documentation lists ML, perceptual, behavioural, time-of-day, manual-override, spike, drift, silence and safety contexts. These should not all be attributed to every build. In practice, context is the set of signals and states available in a particular variant, not semantic understanding of what the viewer is watching.

A “context window” can be understood as a bounded section of history in which the system waits to see whether a change is transient or forms a consistent pattern. This can reduce reactions to brief disturbances, but it adds decision latency. Selecting the window length and confirmation condition requires measurement; this monograph assigns no universal values.

6.3 Decision layers

The logical design can be divided into these stages:

- 1 **Perception:** smooths input and estimates a difference from a reference level, trend, spikes or drift, depending on the variant.
- 2 **Event interpretation:** groups measurements into candidates such as stable behaviour, rise, fall or sudden spike.
- 3 **Context update:** considers history, active mode and available ML flags; it determines whether an event supports an emerging pattern.
- 4 **Intent and comfort assessment:** considers whether a change appears undesirable or remains within the tolerated range.
- 5 **Temporal and safety filters:** dead zone, stable zone, hysteresis, anti-drift, cooldown, anti-spam and command limits constrain oscillation and excessive commands.
- 6 **Decision and execution:** CEPA or application logic may allow VOL_DOWN, VOL_UP or PASS; fuller descriptions also mention blocking or delaying. VolMaster and Broadlink execute a command only after it passes the relevant checks.

Names and stage boundaries differ across diagrams. This model organises shared concepts; it does not replace a source-code specification.

6.4 Zones, dead zone and anti-drift

The behaviour-zone graphic distinguishes stable, transition, unstable and high-risk regions. It is a useful conceptual map: stable input favours PASS, a transition calls for observation, instability may justify confirmation, and a risky pattern calls for a cautious decision or block. Zone names are not automatically numerical thresholds; the supplied materials do not provide a complete set of boundaries for every version.

The **dead zone** is a range of changes that does not trigger a correction. **Hysteresis** uses different conditions for entering and leaving a state, reducing toggling near a boundary. **Anti-drift** is intended to stop slow, cumulative level changes from causing an uncontrolled series of actions. These solve different problems: ignoring small differences, stabilising state changes and controlling long-term trends.

The supplied zone graphic should be read as an illustration of behaviour, not a validated risk scale. Without thresholds, window durations and test results, it cannot establish quantitative algorithm performance.

6.5 Worked decision example

This conceptual example shows why a single class label should not automatically issue a command:

- 1 During dialogue, the level remains within a typical range; the system observes and returns PASS.
- 2 A short spike occurs. If it is too brief or within the dead zone, a filter can ignore it.
- 3 The level then rises gradually. CEPA can open a context window but wait for confirmation.
- 4 If the rise persists and the active model identifies an AD-like pattern, one VOL_DOWN step may be allowed.

- 5 Cooldown and anti-spam prevent a rapid repeat; the next measurement checks whether behaviour is returning to a stable region.
- 6 Once stable, the system closes the context and resumes observation.

This illustrates the process described in project materials. It is not a repeated test result or a guarantee that the public build always responds this way.

6.6 Originality: what the evidence supports

The strongest supported differentiator in the project documents is an **application-level architecture**: separating signal perception, pattern interpretation, behaviour selection and command execution, with explicit PASS states, temporal constraints and context. If maintained in implementation, this goes beyond “AD label → immediate command.”

That does not establish that individual components—classification, spike detection, thresholds, hysteresis, time windows, state machines or cooldowns—are scientifically new. These are established families of techniques. Potential originality lies in their particular organisation for TV volume control, which should be assessed against prior systems and the implementation.

In this monograph, “CEPA originality” means **a distinctive project concept and integration as described in the supplied documentation**, not verified global or patent novelty, nor the result of a market comparison. The [README](#) makes strong uniqueness claims, but the supplied evidence contains no literature review or comparative benchmark that independently confirms them.

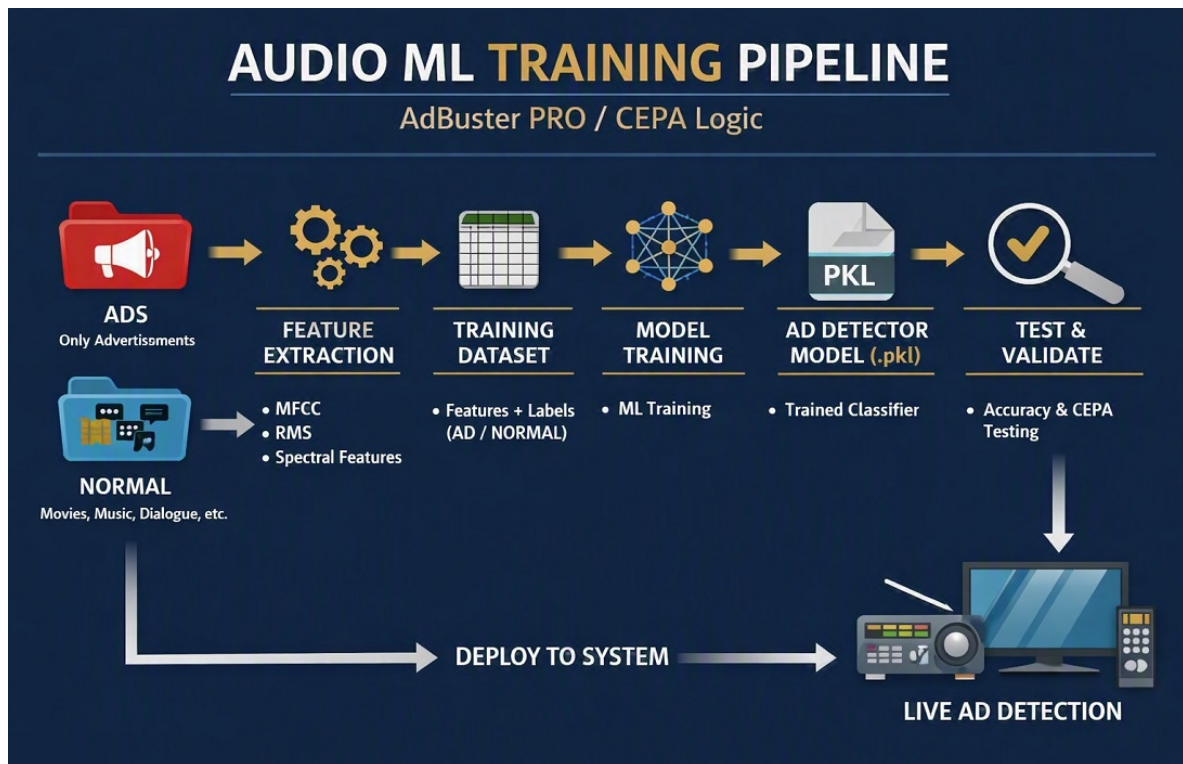
Implementation scope matters as well: according to the README, the public ZIP contains CEPA Core (Perception → Behaviour → Action), while the full CEPA Logic v2 described in the documentation is not included. This section explains the project’s model; without a code audit, each detailed layer, zone and filter cannot be attributed to the released build.

7. Machine learning and data

The public ZIP documentation lists two model files: `model.pkl` and `model_deep.pkl`. Their descriptions assign them short-term and longer-pattern roles, respectively. `ad_detector.pkl` and `data.csv` appear in laboratory materials and are described as experimental, not as part of the public 2.0 package.

Model training is described as an offline process using data collected during or after sessions; training does not happen in real time during audio monitoring. The project may collect derived audio measurements locally for model development. The application does not save raw microphone recordings as WAV files. This describes prototype operation. A separate laboratory pipeline discusses WAV samples as training inputs; that is not the same as the running application saving live microphone audio.

The project materials also describe development from a larger feature set to a lighter four-feature model. This is an account of experiments, not a basis for claiming that every packaged model uses exactly the same inputs.



ML training pipeline

The pipeline separates labelled audio examples, feature extraction, a labelled dataset, training and validation from later deployment. It is a model-preparation cycle outside the volume-control loop. The Data Builder screenshot in the gallery shows a tool for preparing and training data; this should not be confused with online learning while the app monitors audio.

8. Broadlink, VolMaster and IR control

The computer and Broadlink device must communicate over the same local network. The computer may use Wi-Fi or Ethernet, while Broadlink connects over Wi-Fi. After initial device setup, the application uses local communication; internet access is not required for the ordinary operation described by the architecture.

VolMaster is documented as a local Flask server that receives application commands and forwards them to Broadlink. The automatic control layer primarily uses VOL_UP and VOL_DOWN. Other commands, such as MUTE or POWER, are identified as manual in the IR policy description.

Control depends on Broadlink model compatibility, correct device address, IR codes, network connectivity and placement of the transmitter within range of the TV. An HTTP response from the server confirms that the service handled a request; by itself, it does not prove that the TV physically executed the command or that the resulting change was exact.

VolMaster's interface and command controls are shown in the screenshot section. The separate CEPA Setup window configures viewing preferences; it does not configure the Broadlink IP address or IR codes.

9. Privacy and local operation

According to the current project description, the application does not save raw microphone recordings as WAV files and does not upload data to the cloud. Derived audio measurements may be collected locally for model development. The most precise description therefore distinguishes audio recordings from locally stored numerical measurements instead of claiming that no data of any kind is collected.

Local operation does not mean that the system uses no network at all: the PC needs to communicate with Broadlink over the LAN to send an IR command. It means that, according to the documentation, the application does not need

an external cloud service for that communication. Initial Broadlink provisioning may use the official mobile app.

10. Testing, results and evidence limits

The project materials include laboratory screenshots, behaviour reports and example logs. One experimental detector graphic reports accuracy and F1 around 0.80. Its accompanying description identifies the model as an early experiment trained on a small sample. These figures should not be treated as guaranteed performance for the public build in every home or with every advertisement.

The Hybrid v13 report dated 13 September 2026 shows a series of local `/status` responses and `/send` requests returning HTTP 200. This is evidence that the local service responded to requests in the documented excerpt. The log alone does not measure advertisement-detection accuracy or acoustically confirm that every command changed the TV volume.

A reliable model evaluation would require a larger and documented test set, sample counts for each class, per-class results, recording conditions, repeatability and observations on different hardware setups. This publication does not invent missing results.

10.1 Evidence basis

This monograph distinguishes four evidence types. A **package inventory** confirms which files are listed for distribution. The **README and documentation** describe intended architecture and purpose. A **log or interface screenshot** confirms a visible service or application state at a particular time. A **concept diagram** explains an intended flow but does not establish implementation. None of these alone substitutes for a controlled effectiveness test.

An HTTP 200 response is a service-communication result, not a test of CEPA decision quality or a measurement of the TV's response. A plotted screenshot shows a sample of behaviour, but without raw data, labels, protocol and sample counts it cannot support reliable sensitivity or precision estimates. This evidence hierarchy reduces the risk of attributing unmeasured results to the prototype.

10.2 Proposed validation protocol

The following plan is a research recommendation, not a report of completed tests:

- 1 Define event labels, the unit of evaluation and criteria for advertisement start and end; document recording conditions and equipment.
- 2 Split data by complete sessions or recording sources into training, validation and test sets, preventing adjacent fragments of the same audio from leaking across sets.
- 3 Compare at least three variants: a simple threshold with smoothing, the standalone ML classifier, and ML combined with CEPA and safety controls.
- 4 Report precision, recall and F1 for each class, balanced accuracy, false suppressions per hour, missed events, reaction latency and IR command count.
- 5 Measure system outcomes separately: whether a command reached Broadlink, changed the TV level and restored a comfortable level. Acoustic outcome requires an independent sound measurement.
- 6 Run ablations by disabling context, hysteresis, dead zone, cooldown and ML in turn. This estimates each component's contribution instead of assigning the result to the whole system at once.

The protocol should cover different TVs, microphones, distances and room conditions, with repeated trials and uncertainty intervals. Such results would allow comparison between CEPA and a classifier without contextual logic and support a quantitative conclusion about CEPA's contribution.

11. Prototype limitations

- The microphone may capture room sounds as well as TV audio.
- The system evaluates measurements and acoustic patterns; it does not understand programme semantics.
- Loud music, sound effects or dialogue may resemble an unusual audio pattern.

- Actual behaviour depends on the microphone, settings, room acoustics, TV, IR codes and Broadlink device.
- IR commands require correct setup, LAN connectivity and a clear signal path to the device.
- The documentation describes a prototype and experiments, not independent certification or guaranteed production operation.
- Tests on one device or in one session do not establish compatibility with every TV or robustness across all content.

AdBuster operates as a volume-control system, not as a digital compressor connected to the audio path. It changes volume through external IR commands rather than processing and modifying the TV's output audio signal.

12. Startup and requirements

The public build is described as a portable Windows package: download the ZIP, extract it and run `Start.bat`. Automatic control requires a compatible Broadlink IR transmitter; audio monitoring requires a microphone. The computer and IR device must be on the same local network.

The user documentation describes initial Broadlink Wi-Fi setup, checking the device address and IR codes, and placing the transmitter within range of the controlled device. Exact compatibility depends on the model and configuration. Keep the packaged files together because `Start.bat` launches cooperating components.

13. Development status

The public 2.0 package is documented as containing compiled components, two model files, configuration and supporting resources. Source code is not included.

The laboratory track documents an experimental advertisement detector, a dataset-building pipeline, CEPA logic variants and Hybrid Engine v13. These materials describe development directions and should not automatically be treated as features shipped in the test build.

The roadmap lists further work on stability, collection and evaluation of behavioural data, CEPA development and device integration. Planned features are not presented here as already delivered.

14. Licensing, feedback and documentation

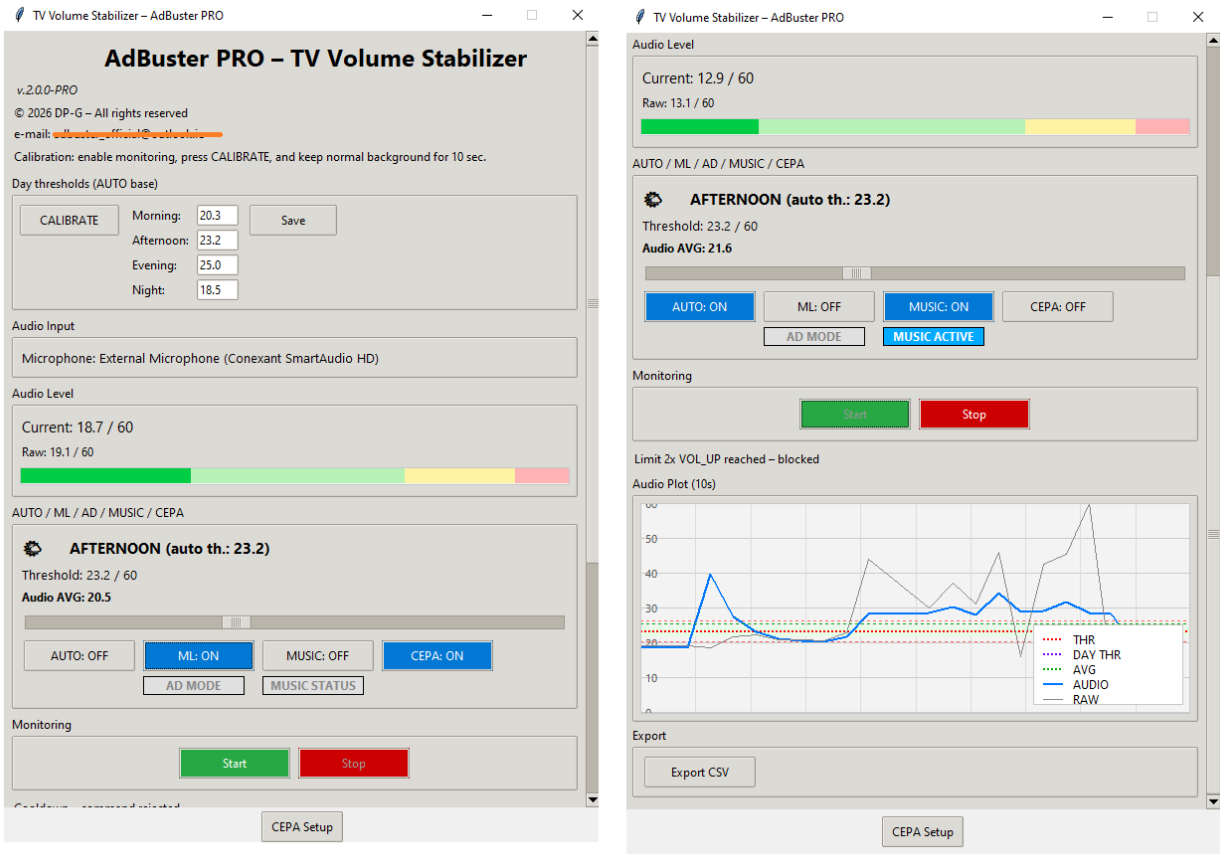
AdBuster PRO is a proprietary project distributed under a PRO license. The public package contains compiled files rather than source code. CEPA Logic has a separate license. Check the current repository license files for applicable terms; this monograph does not replace them.

The test build is intended for personal evaluation and feedback under the applicable license. Useful feedback can include the microphone, TV and Broadlink models, network arrangement, selected mode and observed behaviour. Avoid sharing private configuration values or activation keys.

15. Project graphics and screenshots

The following screenshots show the interfaces supplied with the project materials: the AdBuster PRO monitor, its audio-level graph and mode states, the VolMaster server and a combined view of the running applications. The diagrams and infographic illustrate concepts described in the documentation.

AdBuster PRO — monitor and audio graph



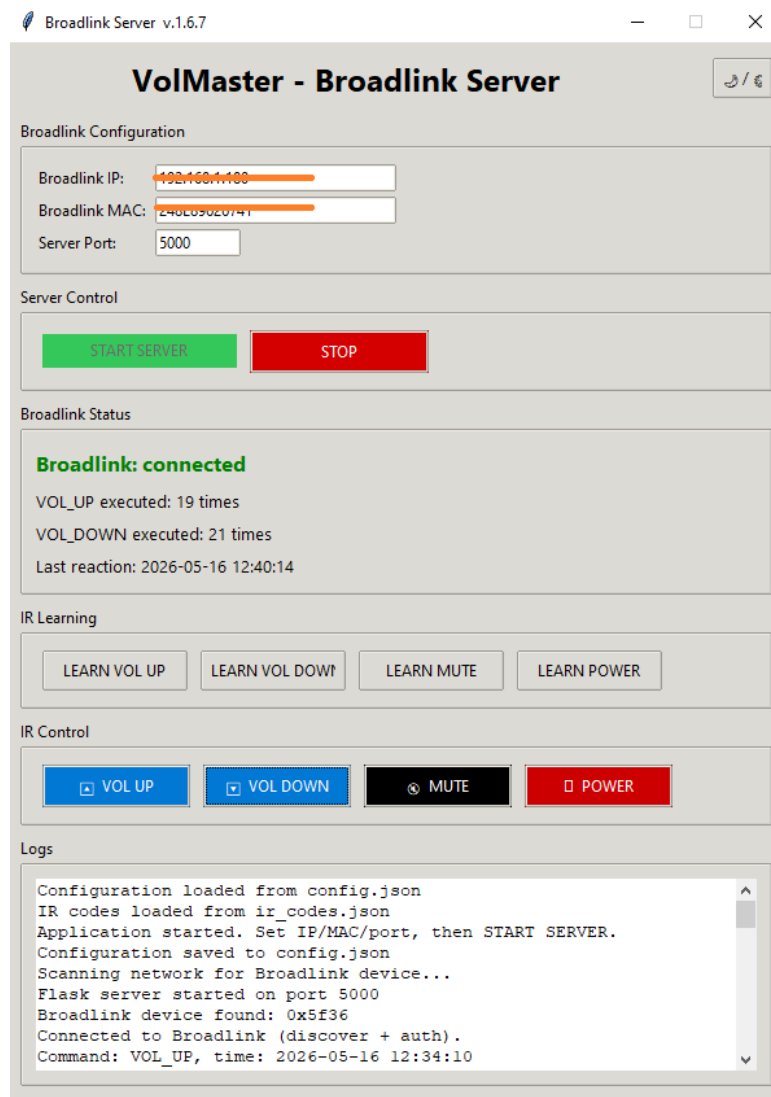
Two AdBuster PRO screenshots showing settings and audio monitoring

The first view shows the main screen with current and raw audio levels, time-of-day threshold profiles and AUTO, ML, MUSIC and CEPA switches. The second shows a monitoring panel with an approximately ten-second plot, series legend and a message that the VOL_UP limit was reached. The displayed values and statuses are the state at capture time; they are not a comparative test of detection sensitivity or effectiveness.

The panel distinguishes the user-facing level from the raw measurement and shows an average and a reference threshold. This matters because a decision based on a smoothed value may be less sensitive to a single impulse than a decision based on an instantaneous maximum. The plot labels THR, DAY THR, AVG, AUDIO and RAW identify different series for thresholds and observed levels; their exact calculation requires documentation for the specific build.

The screenshots also show different switch configurations: ML is active in one, while AUTO and MUSIC are active in the other. They should not be read as one simultaneous application state. The VOL_UP limit message illustrates a safety constraint; it does not prove that an advertisement was detected or that the volume correction was acoustically correct.

VolMaster — Broadlink server

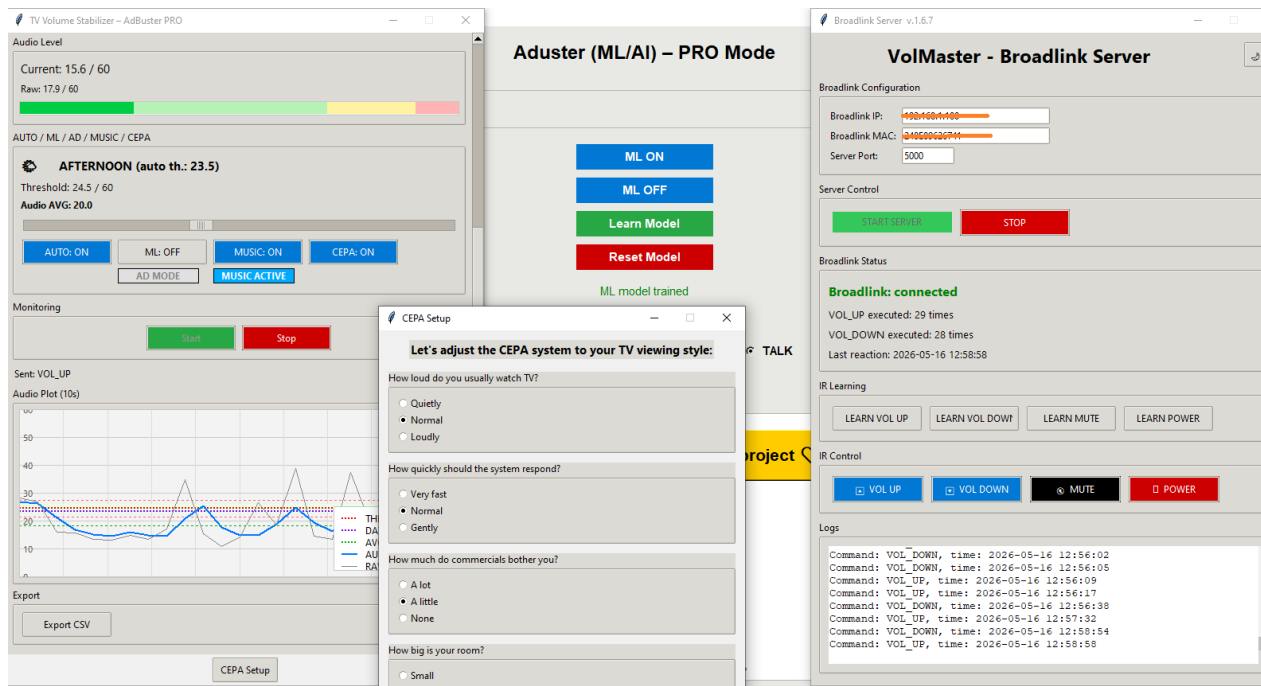


VoIMaster server and IR control window

The panel separates Broadlink configuration, server control, connection status, IR-code learning, manual buttons and command logs. The screenshot shows an example “connected” status and command counters, but the interface alone does not confirm that the TV received every transmission. Network identifiers are obscured in the image.

The visible log shows the server loading configuration, finding a device and recording VOL_UP and VOL_DOWN commands with timestamps. It illustrates the transport layer: the application makes a decision, the local service accepts a request and a device code is sent over IR. The screenshot contains no measurement of the TV’s resulting loudness or confirmation that its IR receiver accepted each pulse; those require separate observation.

Applications running together

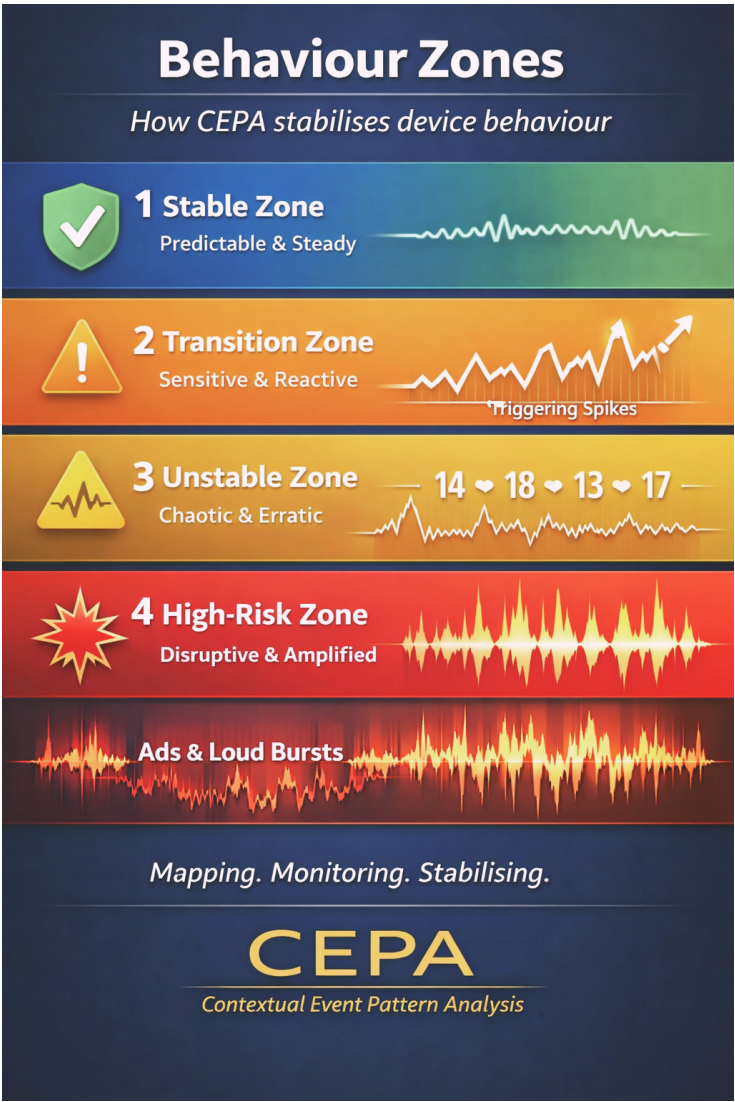


AdBuster PRO, ML controller, CEPA Setup and VolMaster

The composition brings together several separate windows: the AdBuster monitor, ML panel, CEPA Setup preference wizard and VolMaster server. It illustrates the division of the interface across processes and tools; it is not one unified screen or proof that all displayed switches were active in the same run.

Architecturally, the ML panel presents training functions and model state, VolMaster focuses on the IR device, and the main window focuses on audio and operating modes. This supports a separation between classification, decision and execution. The window layout alone does not establish process isolation, API boundaries or independent failure modes; those require inspection of the package and inter-process logs.

CEPA behaviour zones

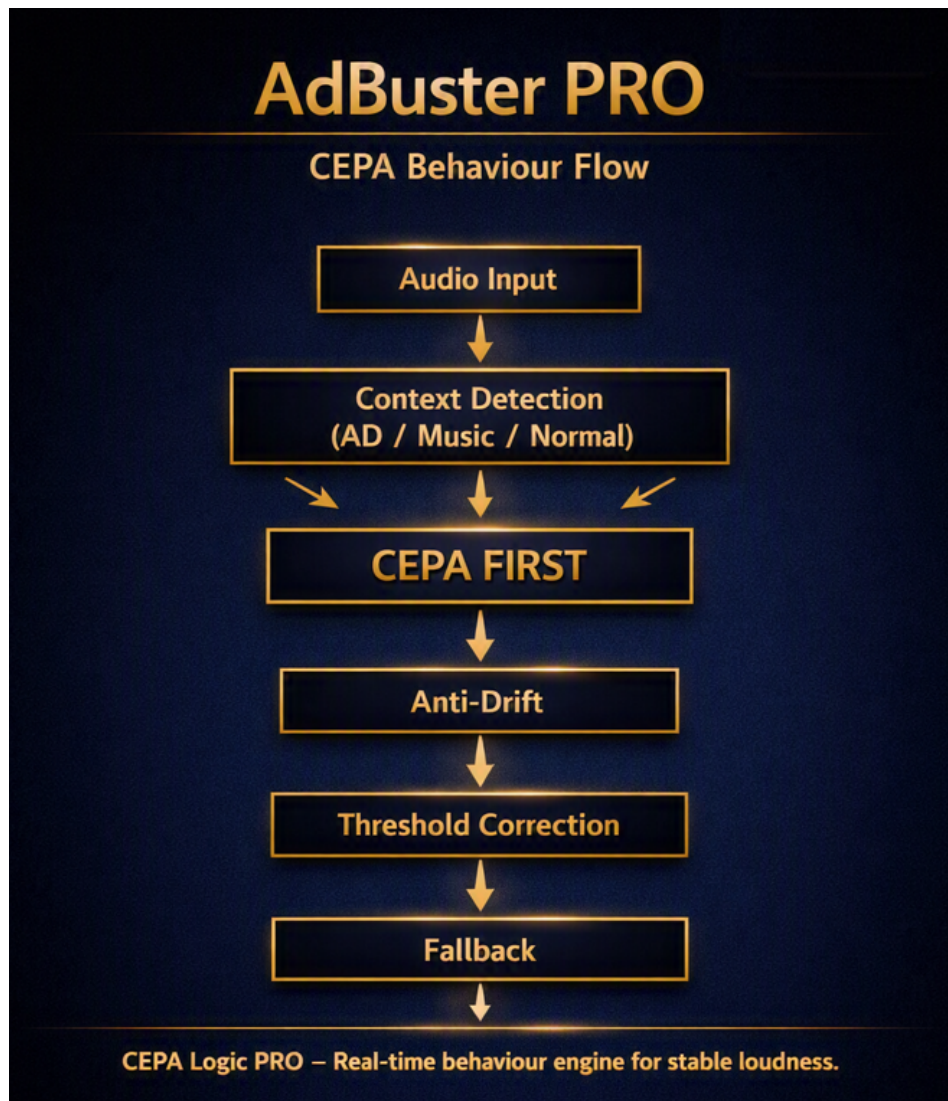


Infographic of stable, transition, unstable and high-risk zones

The board shows four qualitative states—stable, transition, unstable and high risk—plus a separate example of loud advertisements and short bursts. Its purpose is to explain the behaviour vocabulary. It contains no calibration, numerical boundaries or results that would validate the zones as a risk scale.

Operationally, the zones suggest graduated reactions: in a stable region the system should leave normal dynamics alone; during transition it should gather more context; under instability it should check whether a pattern persists; and in a high-risk state it may constrain further level increases. This interprets the diagram’s intent rather than asserting confirmed runtime rules. An implementation should publish boundaries, state-duration requirements and behaviour at zone transitions.

CEPA behaviour flow

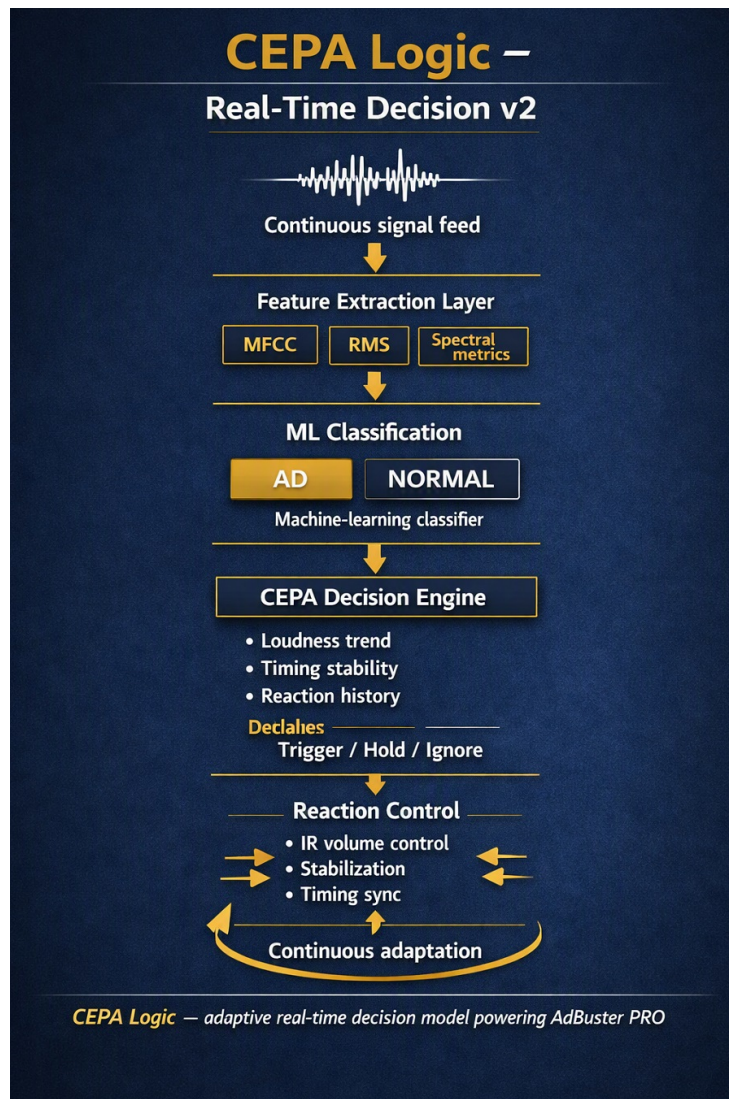


AdBuster PRO behaviour flow diagram

The diagram summarises the sequence: audio input, AD/MUSIC/TALK/NORMAL context detection, CEPA priority, anti-drift, threshold correction and fallback. It highlights that a response is intended to consider state and limits, not only a model label. This is a conceptual diagram, not an execution trace or complete algorithm specification.

Anti-drift and threshold correction appear before fallback, which can be read as an attempt to stabilise the observation before applying a substitute behaviour. The graphic does not define exact transition conditions. The distinction between diagram and algorithm matters: reproducing a decision would require parameters, input state, measurement history and unambiguous rules.

Real-time decision process



CEPA real-time decision layers diagram

This board expands the decision layer: input features, AD/NORMAL classification, reaction history and a trigger/hold/ignore choice, followed by IR control and stabilisation. Together with the layer description in Chapter 6, it shows ML as a source of context and control as a separate responsibility. Names and completeness vary by version; the graphic does not prove that the entire illustrated path is in the public build.

The “Continuous adaptation” arrow suggests a loop in which the system keeps observing the acoustic result after a command and may adjust later decisions. In the documented arrangement, feedback comes from the microphone rather than a digital readback of the TV’s current volume. If the TV does not execute a command, or changes the level differently, the audio layer can observe only the acoustic consequence, with delay and room effects.

16. Glossary

- **RMS** — a numerical measure of signal energy/level over a defined window.
- **STD** — a measure of the spread of observed values.
- **DELTA** — change relative to an earlier observation.
- **RANGE** — the span of values within an analysis window.
- **CEPA** — Contextual Event Pattern Analysis; the context and decision layer.
- **IR** — infrared, used here for remote-control-like commands.
- **VolMaster** — the local server that forwards commands to Broadlink.
- **AD Detector** — an experimental model documented outside the public 2.0 ZIP.

